

LAB 1: Recognizing Cyber Threats Using Simple AI Rules

Lab Overview	Details
Mapped Course Outcome(s)	CO1, CO2
Bloom's Level	L2 - Understand / L3 - Apply
Total Marks	50
Focus Areas	Rule-based AI, phishing, malware, ransomware, risk scoring, privacy and human oversight
Platform	Google Colab / Python
Duration	4 hours

Problem Statement

Use a small faculty-provided CSV containing safe and suspicious messages. First identify messages manually, then build a transparent keyword-based system that classifies each message as Safe, Phishing, Malware or Ransomware. The system must assign a risk level, recommend a safe action and document privacy, false-alarm and human-review considerations.

Lab Instructions

- Use simple_cyber_messages.csv or the faculty-provided starter notebook.
- Run cells sequentially and complete only the marked TODO sections.
- Use sanitized sample messages only; never click or open real suspicious links/files.
- Record output after every major step and write observations in simple language.
- Demonstrate at least one new message during viva.

Learning Outcomes

- Explain how transparent AI rules can support cyber-security analysts.
- Identify common indicators of phishing, malware and ransomware.
- Create and test a keyword-based classifier.
- Calculate simple accuracy and recognize false positives.
- Apply privacy, accountability and human-review principles.

General Setup

PYTHON / GOOGLE COLAB SKELETON

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score

df = pd.read_csv('simple_cyber_messages.csv')
display(df.head())
print('Dataset shape:', df.shape)
```

Q1. Manual Threat Identification (Marks: 5)

AIM

To manually classify ten sample messages and identify the words that influenced each decision.

Theory / Concept

- Phishing attempts commonly use urgency, account verification, password, OTP, bank or prize language.
- Malware messages refer to infected files, viruses, trojans, spyware, executables or suspicious attachments.
- Ransomware messages mention encrypted or locked files, ransom, bitcoin, payment or restoration demands.
- A message should be treated as Safe only when no clear threat indicator is present and the context is legitimate.

Algorithm / Guided Procedure

1. Open the first ten messages in the dataset.
2. Read each message without opening any link or attachment.
3. Assign one category: Safe, Phishing, Malware or Ransomware.
4. Write the important words/phrases that influenced the decision.
5. Compare manual labels with the correct_category column.

PYTHON / GOOGLE COLAB SKELETON

```
manual_table = df.head(10)[
    ['message_id', 'message', 'correct_category']
].copy()

# TODO: Add important words for each of the ten messages
manual_table['important_words'] = [
    'meeting, scheduled',
```

```
'suspended, click here, verify, password',
urgently',
'executable, trojan, infect',
'encrypted, bitcoin, restore files',
'library, returned',
'prize, login, claim',
'spyware, attachment',
'locked files, ransom, payment',
'official portal',
'urgent, bank, verify, OTP'
]

display(manual_table)
```

SAMPLE / EXPECTED OUTPUT

A ten-row table showing message ID, message text, correct category and important words.

Points to Record in Lab File

- Completed manual-classification table.
- At least two indicators for each threat message.
- One sentence explaining why safe messages should still be checked in context.

Q2. Keyword Lists and Rule-Based Classifier (Marks: 10)

AIM

To create transparent keyword lists and a Python function that predicts the category of a message.

Theory / Concept

- Rule-based AI follows conditions explicitly written by a human.
- The classifier counts matching indicators for each threat category.
- The category with the highest score is selected; no match means Safe.
- Transparent rules are easy to explain but may fail when attackers use unfamiliar wording.

Algorithm / Guided Procedure

1. Create keyword lists for Phishing, Malware and Ransomware.
2. Convert the message to lowercase.
3. Count the matched keywords in each list.
4. Return Safe when every score is zero.
5. Otherwise return the highest-scoring category, matched words and score.

PYTHON / GOOGLE COLAB SKELETON

```

RULE_KEYWORDS = {
    'Phishing': [
        'click here', 'verify', 'password', 'urgent',
        'otp', 'prize', 'bank', 'login', 'blocked'
    ],
    'Malware': [
        'malware', 'virus', 'infected', 'attachment',
        'trojan', 'spyware', 'executable'
    ],
    'Ransomware': [
        'encrypted', 'ransom', 'bitcoin', 'restore files',
        'locked files', 'unlock', 'payment demand'
    ]
}

def classify_message(text):
    text = str(text).lower()
    scores = {
        category: sum(word in text for word in words)
        for category, words in RULE_KEYWORDS.items()
    }
    predicted = max(scores, key=scores.get)

    if scores[predicted] == 0:
        return 'Safe', '', 0

    matched = [
        word for word in RULE_KEYWORDS[predicted]
        if word in text
    ]
    return predicted, ', '.join(matched), scores[predicted]

print(classify_message(
    'Urgent: click here to verify your password'
))

```

SAMPLE / EXPECTED OUTPUT

Example: ('Phishing', 'click here, verify, password, urgent', 4)

Points to Record in Lab File

- Final keyword lists.
- Output of at least four test messages, including one Safe example.
- A short limitation: keyword rules may miss synonyms or new attack language.

Q3. Complete Dataset Classification and Accuracy (Marks: 10)

AIM

To run the classifier on the complete dataset and calculate correct predictions and accuracy.

Theory / Concept

- Accuracy is the proportion of correct predictions among all predictions.
- A high score on a small classroom dataset does not prove real-world effectiveness.
- A false positive is a safe message incorrectly labelled as a threat.
- A false negative is a threat incorrectly labelled as safe.

Algorithm / Guided Procedure

1. Apply `classify_message` to every row.
2. Create `predicted_category`, `matched_keywords` and `keyword_score` columns.
3. Compare `predicted_category` with `correct_category`.
4. Count correct and incorrect predictions.
5. Calculate accuracy as a percentage.

Algorithm: Message Classification and Accuracy Calculation

Input: A DataFrame `df` containing messages and their correct categories.

Output: Predicted categories, matched keywords, scores, and classification accuracy.

1. Start.
2. Create three empty lists to store:
 - Predicted categories
 - Matched keywords
 - Keyword scores
3. Read each message from the message column.
4. Pass the message to the `classify_message()` function.
5. Receive the following results:
 - Predicted category
 - Matched keywords
 - Keyword score
6. Add these results to their respective lists.
7. Repeat Steps 3–6 until all messages are classified.
8. Add the prediction results as new columns in the DataFrame.
9. Compare the `correct_category` with the `predicted_category` for every message.
10. Store True if both categories match; otherwise, store False.
11. Count the number of correct predictions.
12. Calculate incorrect predictions using:
13. Calculate accuracy using:

14. Display the message ID, correct category, predicted category, matched keywords, and prediction result.
15. Print correct predictions, incorrect predictions, and accuracy.
16. Stop.

PYTHON / GOOGLE COLAB SKELETON

```

results = df['message'].apply(classify_message)

df[
    ['predicted_category', 'matched_keywords', 'keyword_score']
] = pd.DataFrame(results.tolist(), index=df.index)

df['correct'] = (
    df['correct_category'] == df['predicted_category']
)

accuracy = accuracy_score(
    df['correct_category'],
    df['predicted_category']
)

display(df[[
    'message_id', 'correct_category',
    'predicted_category', 'matched_keywords', 'correct'
]])

print('Correct predictions:', df['correct'].sum())
print('Incorrect predictions:', (~df['correct']).sum())
print(f'Accuracy: {accuracy * 100:.2f}%')
'''

# Create empty lists to store the prediction results.
predicted_categories = []
matched_keywords_list = []
keyword_scores = []

# Read each message from the 'message' column.
for message in df['message']:

    # Classify the current message.
    # The function returns category, matched keywords, and score.
    category, matched_keywords, score = classify_message(message)

    # Store the predicted category.
    predicted_categories.append(category)

    # Store the matched keywords.
    matched_keywords_list.append(matched_keywords)

    # Store the keyword score.
    keyword_scores.append(score)

# Add predicted categories as a new DataFrame column.

```

```

df['predicted_category'] = predicted_categories

# Add matched keywords as a new DataFrame column.
df['matched_keywords'] = matched_keywords_list

# Add keyword scores as a new DataFrame column.
df['keyword_score'] = keyword_scores

# Create a column to check whether each prediction is correct.
df['correct'] = (
    df['correct_category'] == df['predicted_category']
)

# Count the total number of correct predictions.
correct_predictions = df['correct'].sum()

# Count the total number of incorrect predictions.
incorrect_predictions = len(df) - correct_predictions

# Calculate accuracy.
accuracy = correct_predictions / len(df)

# Select the columns that we want to display.
output_columns = [
    'message_id',
    'correct_category',
    'predicted_category',
    'matched_keywords',
    'correct'
]

# Display the selected columns.
display(df[output_columns])

# Print the number of correct predictions.
print('Correct predictions:', correct_predictions)

# Print the number of incorrect predictions.
print('Incorrect predictions:', incorrect_predictions)

# Print accuracy as a percentage with two decimal places.
print(f'Accuracy: {accuracy * 100:.2f}%')
'''

```

SAMPLE / EXPECTED OUTPUT

For the supplied classroom dataset, the expected result is approximately 28 correct predictions out of 28 (100%). Real messages are more difficult.

Points to Record in Lab File

- Complete result table.
- Correct and incorrect prediction counts.
- Accuracy percentage.

- One paragraph explaining why classroom accuracy can be optimistic.

Q4. Risk Level, Recommended Action and Bar Chart (Marks: 10)

AIM

To convert the predicted category and keyword score into a simple risk response and summary chart.

Theory / Concept

- Safe messages are assigned Low risk.
- Ransomware or multiple warning indicators are treated as High risk.
- Other detected threats are treated as Medium risk.
- Actions should be cautious and should direct serious cases to a human analyst.

Algorithm / Guided Procedure

1. Create a function that returns risk level and recommended action.
2. Apply it to every classified message.
3. Count predictions in each category.
4. Plot a labelled bar chart.
5. Test one new input live.

Algorithm

1. Start.
2. Create empty lists for risk levels and recommended actions.
3. Read each row from the DataFrame.
4. Get its predicted category and keyword score.
5. If the category is Safe:
 - Set risk level to Low.
 - Recommend normal caution.
6. Otherwise, if the category is Ransomware or the keyword score is 2 or more:
 - Set risk level to High.
 - Recommend avoiding the message, isolating the system if required, and reporting it.
7. Otherwise:
 - Set risk level to Medium.
 - Recommend verifying the sender and reporting the message.
8. Store the risk level and recommended action.
9. Repeat until all rows are processed.
10. Add the stored results to the DataFrame.
11. Count the messages in each predicted category.
12. Arrange the categories as Safe, Phishing, Malware, and Ransomware.

13. Plot the category counts using a bar graph.
14. Display the graph.
15. Stop.

PYTHON / GOOGLE COLAB SKELETON

```
def add_risk_and_action(row):
    if row['predicted_category'] == 'Safe':
        return 'Low', 'Continue normal caution.'

    if (
        row['predicted_category'] == 'Ransomware'
        or row['keyword_score'] >= 2
    ):
        return (
            'High',
            'Do not click/open; isolate if required and report.'
        )

    return (
        'Medium',
        'Do not interact; verify the sender and report.'
    )

risk_output = df.apply(add_risk_and_action, axis=1)
df[['risk_level', 'recommended_action']] = pd.DataFrame(
    risk_output.tolist(), index=df.index
)

order = ['Safe', 'Phishing', 'Malware', 'Ransomware']
df['predicted_category'].value_counts().reindex(
    order, fill_value=0
).plot(kind='bar')

plt.title('Predicted Message Categories')
plt.xlabel('Category')
plt.ylabel('Number of Messages')
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

"""
# Create empty lists to store risk levels and recommended actions.
risk_levels = []
recommended_actions = []

# Read each row of the DataFrame one by one.
for index, row in df.iterrows():

    # Get the predicted category of the current message.
    category = row['predicted_category']

    # Get the keyword score of the current message.
```

```

score = row['keyword_score']

# If the message is safe, assign a low risk level.
if category == 'Safe':
    risk = 'Low'
    action = 'Continue normal caution.'

# If the message is ransomware or has two or more risky keywords,
# assign a high risk level.
elif category == 'Ransomware' or score >= 2:
    risk = 'High'
    action = 'Do not click/open; isolate if required and report.'

# Assign medium risk to all other suspicious messages.
else:
    risk = 'Medium'
    action = 'Do not interact; verify the sender and report.'

# Store the risk level in the list.
risk_levels.append(risk)

# Store the recommended action in the list.
recommended_actions.append(action)

# Add the risk levels as a new DataFrame column.
df['risk_level'] = risk_levels

# Add the recommended actions as a new DataFrame column.
df['recommended_action'] = recommended_actions

# Set the order in which categories should appear in the graph.
category_order = ['Safe', 'Phishing', 'Malware', 'Ransomware']

# Count the number of messages in each predicted category.
category_counts = df['predicted_category'].value_counts()

# Arrange the categories in the required order.
# If a category is missing, its count will be set to 0.
category_counts = category_counts.reindex(
    category_order,
    fill_value=0
)

# Create a bar graph of the predicted message categories.
category_counts.plot(
    kind='bar',
    color='skyblue',
    edgecolor='black'
)

# Add a title to the graph.

```

```
plt.title('Predicted Message Categories')

# Add a label to the horizontal axis.
plt.xlabel('Category')

# Add a label to the vertical axis.
plt.ylabel('Number of Messages')

# Keep the category labels horizontal.
plt.xticks(rotation=0)

# Adjust spacing so that all graph elements are visible.
plt.tight_layout()

# Display the graph.
plt.show()
'''
```

SAMPLE / EXPECTED OUTPUT

A table containing category, risk and recommended action, plus a bar chart for Safe, Phishing, Malware and Ransomware.

Points to Record in Lab File

- Risk/action table.
- Labelled bar chart.
- New-message demonstration result.
- Explanation of why High-risk results require verification.

Q5. Responsible Use, Reflection and File Saving (Marks: 5)

AIM

To document privacy, bias, false alarms, accountability and human-review requirements and save the final result.

Theory / Concept

- Cyber data may contain names, email addresses, account IDs and phone numbers.
- Keyword lists may work better for some message styles or languages than others.
- False alarms can disrupt users; missed attacks can cause harm.
- The person using the system remains responsible for the final decision.

Algorithm / Guided Procedure

1. Write one point each on privacy, bias, false alarms, accountability and human review.
2. Save the completed result table as Lab1_Final_Result.csv.

3. Write a concise conclusion describing the value and limitation of rule-based AI.

PYTHON / GOOGLE COLAB SKELETON

```
responsible_use = [
    'Privacy: remove personal identifiers before analysis.',
    'Bias: keyword lists may not cover every writing style.',
    'False alarms: safe messages may contain warning words.',
    'Accountability: humans remain responsible for action.',
    'Human review: high-risk or unclear cases must be checked.'
]

for point in responsible_use:
    print('-', point)

df.to_csv('Lab1_Final_Result.csv', index=False)
print('Saved: Lab1_Final_Result.csv')
```

SAMPLE / EXPECTED OUTPUT

Five responsible-use points and a saved CSV file.

Points to Record in Lab File

- Responsible-use note.
- Saved CSV evidence.
- Five-to-eight line conclusion.

Assessment Scheme

Question / Component	Evaluation Focus	Marks
Q1	Manual classification and important words	5
Q2	Keyword lists and classifier function	10
Q3	Complete testing, correctness and accuracy	10
Q4	Risk, action and category chart	10
Q5	Responsible use, reflection and saved file	5
Lab 1 Assessment	Correctness, code quality, documentation, demonstration and viva	10
Total		50

Submission Checklist

Required Evidence	Check
Executed and well-commented Colab notebook.	☐
Manual classification table for ten messages.	☐

Required Evidence	Check
Complete prediction and accuracy table.	☐
Risk/action table and labelled bar chart.	☐
Responsible-use note and conclusion.	☐
Lab1_Final_Result.csv and shareable Colab link.	☐